Automating IxNetwork

Hubert Gee

Solution Architect – Automation



Automating IxNetwork

WITH RESTPY AND API BROWSER

Agenda

IXNETWORK AUTOMATION TOOLS

- API Browser
- RestPy: SDK for script development
- Sample Scripts

IxNetwork Automation Platforms

MULTIPLE CHOICES TO FIT YOUR ENVIRONMENT

Workstation to run scripts from

- Windows
- Linux

IxNetwork API Server Options	Platforms & Benefits
Windows API Server	• Windows 10 • Single session • GUI access
Windows Connection Manager	• Windows Server • Up to 10 sessions • GUI access
Linux API Server	• OVA, QCOW2, chassis and Docker • Up to 40 sessions • Web GUI access

Chassis Options

- Windows Chassis
- Linux Chassis
- Virtual Chassis (KVM, vmware)

RESTPy

What is it?

- Python SDK for automating IxNetwork using ReST APIs.

Benefits

- Python classes and attributes that reflects the IxNetwork API tree
- Don’t need to know IxNetwork API syntaxes
- Exception error handlings

Prerequisites

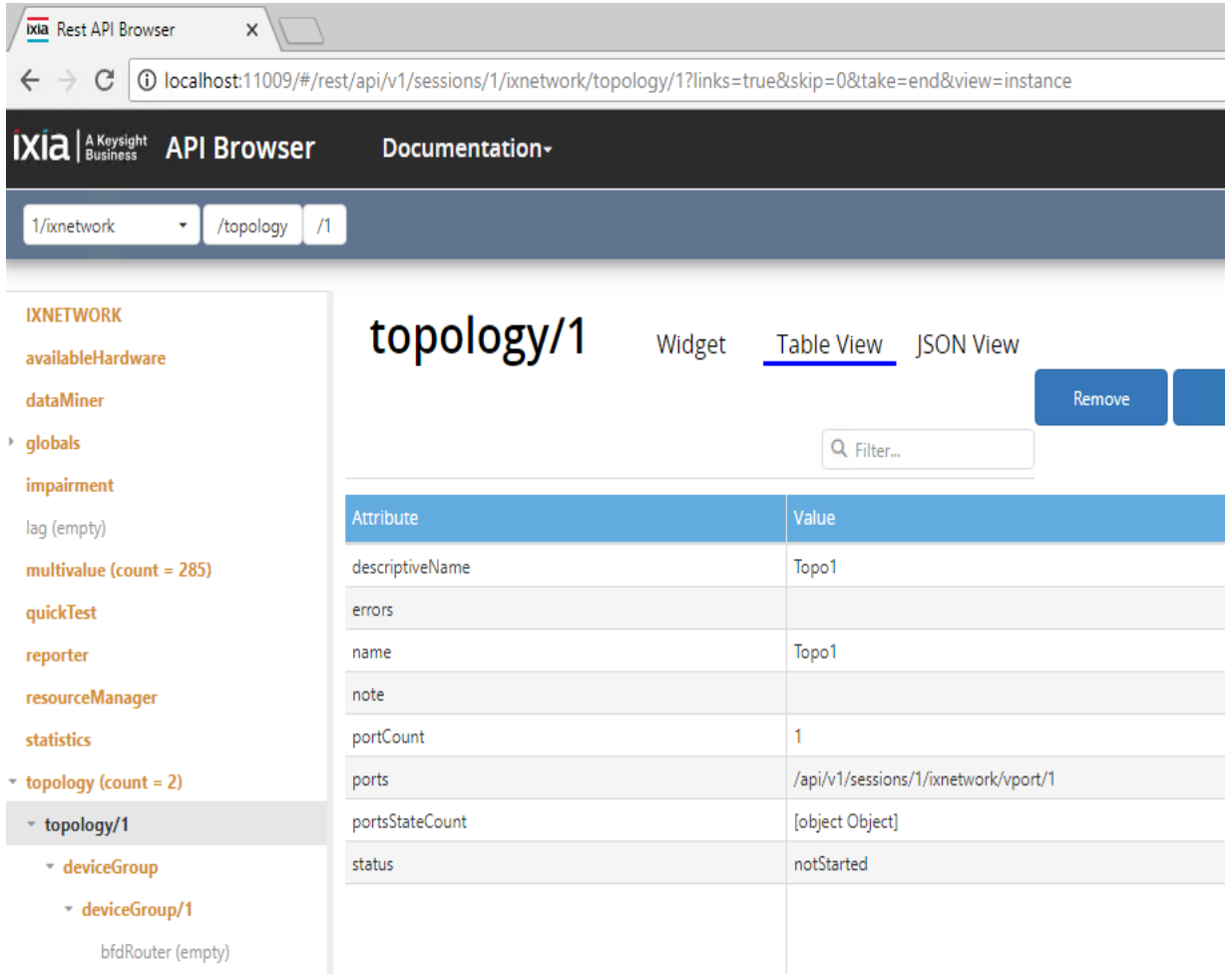
UNDERSTANDING THE IXNETWORK API BROWSER

How to open the API browser:

- For Windows IxNetwork GUI:
  - Go to File, Tools and select IxNetwork API Browser
  - On a browser: https://<ip>:11009
- From web UI:
  - Enter a session and click on the </> icon

- The left pane shows all the API entry nodes.

- Click on any node to view or modify the attributes on the right.



How to Connect

IMPORT THE RESTPY LIBRARY FOR SCRIPTING

```
1
2   # Import the RESTPy package
3   from ixnetwork_restpy import *
4
5   session = SessionAssistant(IpAddress='192.168.70.12')
6
7   ixNetwork = session.ixnetwork
8
9
10
11
```

At this point, use the ***ixNetwork*** object as the entry point to the IxNetwork API tree.

How RestPy Reflects the API Browser

print(ixNetwork.Globals)

Attributes and values printed

Globals[-1]: /api/v1/sessions/1/ixnetwork/globals

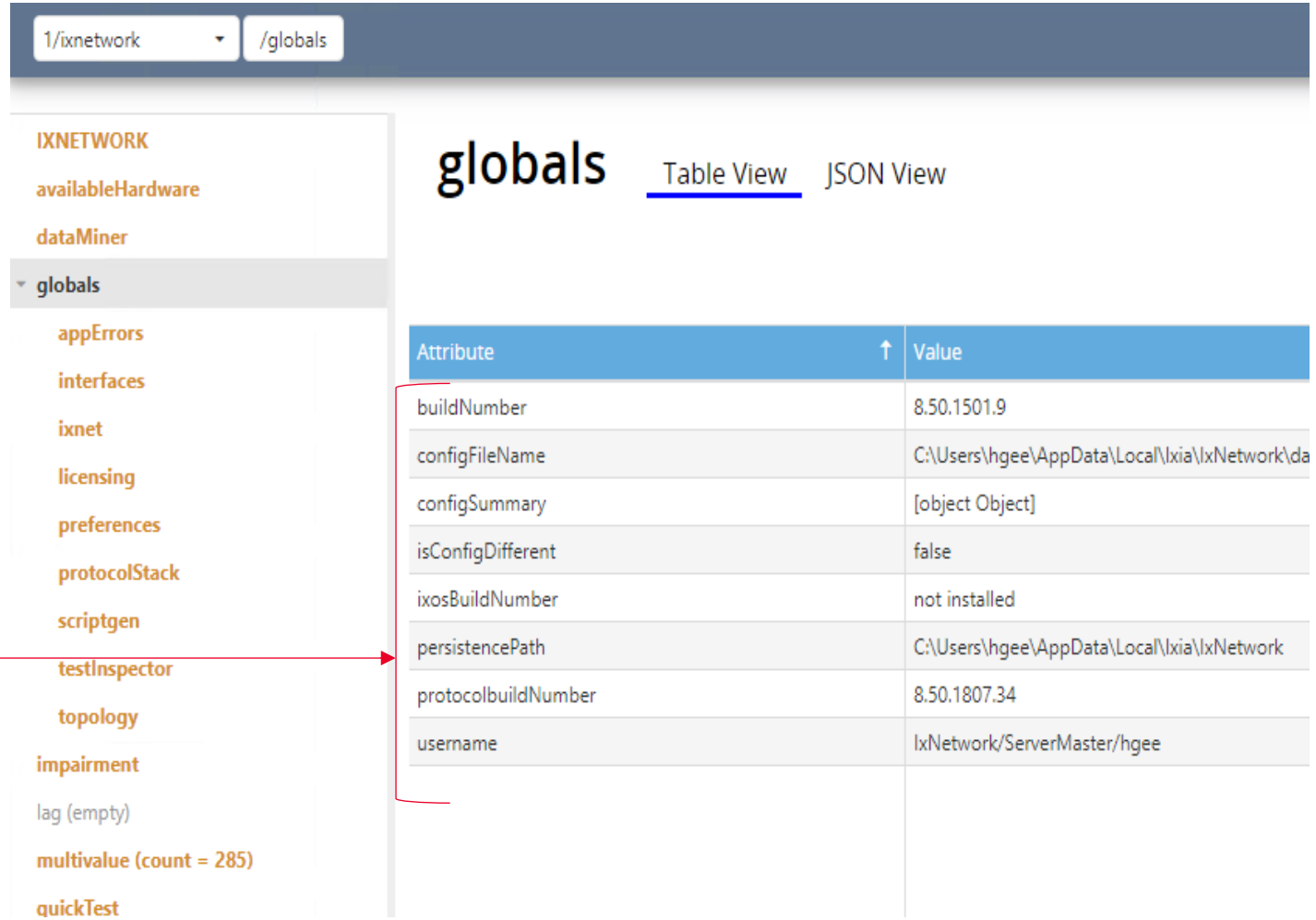
BuildNumber: 8.50.1501.9

IsConfigDifferent: False

IxosBuildNumber: not installed

ProtocolbuildNumber: 8.50.1807.34

Username: IxNetwork/ServerMaster/hgee



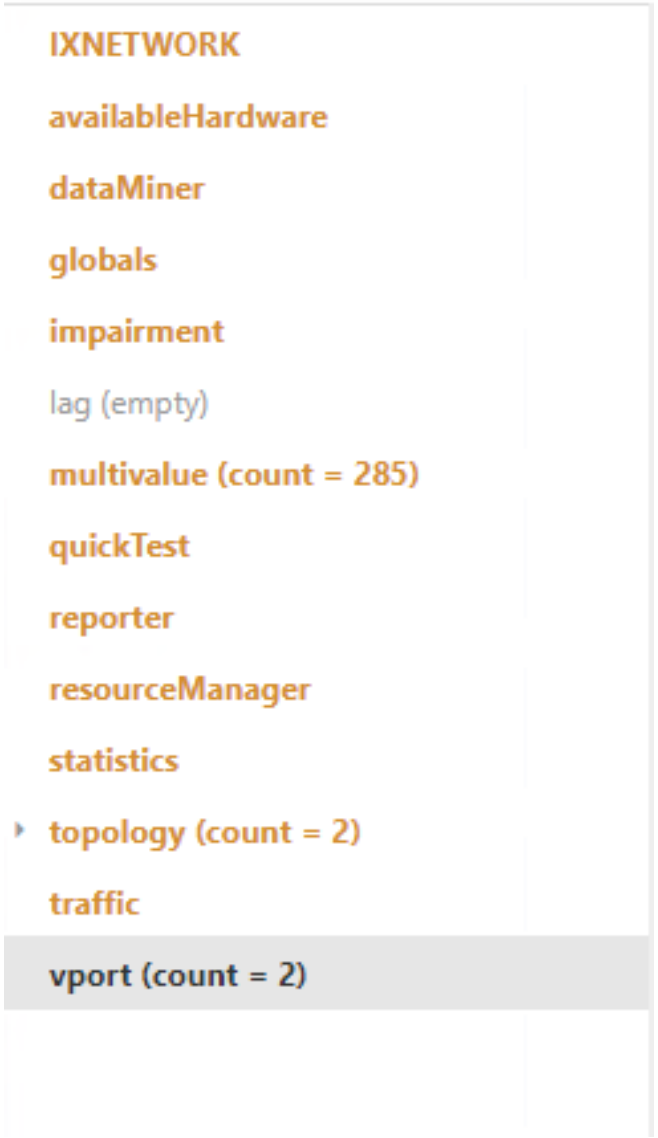
Create Virtual Ports and Assign To Physical Ports

WITH PORTMAP ASSISTANT

```
portMap = session.PortMapAssistant()

vPort1 = portMap.Map(IpAddress='192.168.70.128', CardId=1, PortId=1, Name='Port_1')
vPort2 = portMap.Map(IpAddress='192.168.70.128', CardId=1, PortId=2, Name='Port_2')

portMap.Connect(ForceOwnership=True)
```



How To View Object Attributes

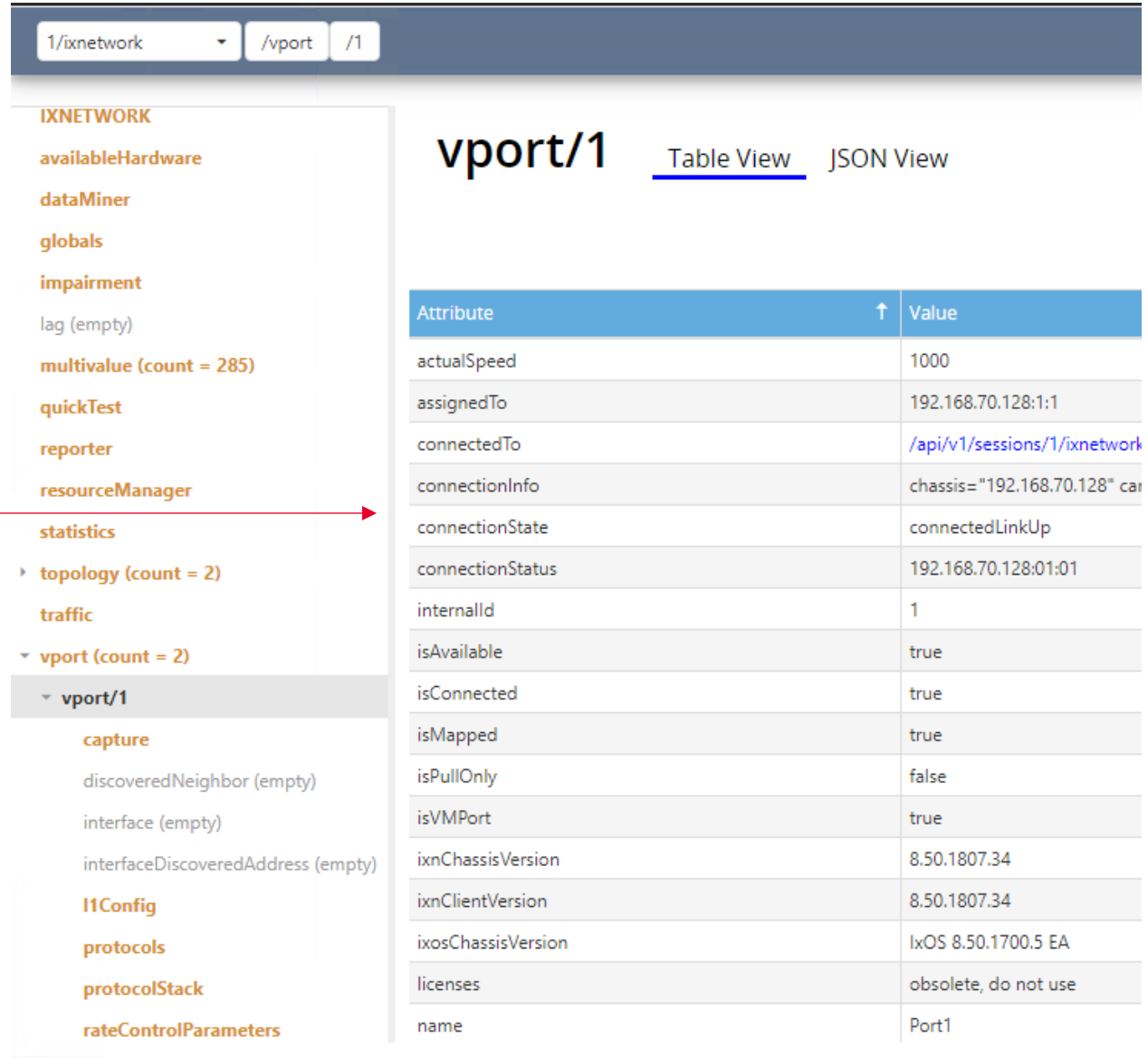
print(vPort1)

Vport[-1]: /api/v1/sessions/1/ixnetwork/vport/1
- ActualSpeed: 1000
- AssignedTo: 192.168.70.128:1:1
- ConnectionState: connectedLinkUp
- ConnectionStatus: 192.168.70.128:01:01
- IsAvailable: True
- IsConnected: True
- Name: Port1
- State: up

Working with the vport1 attributes

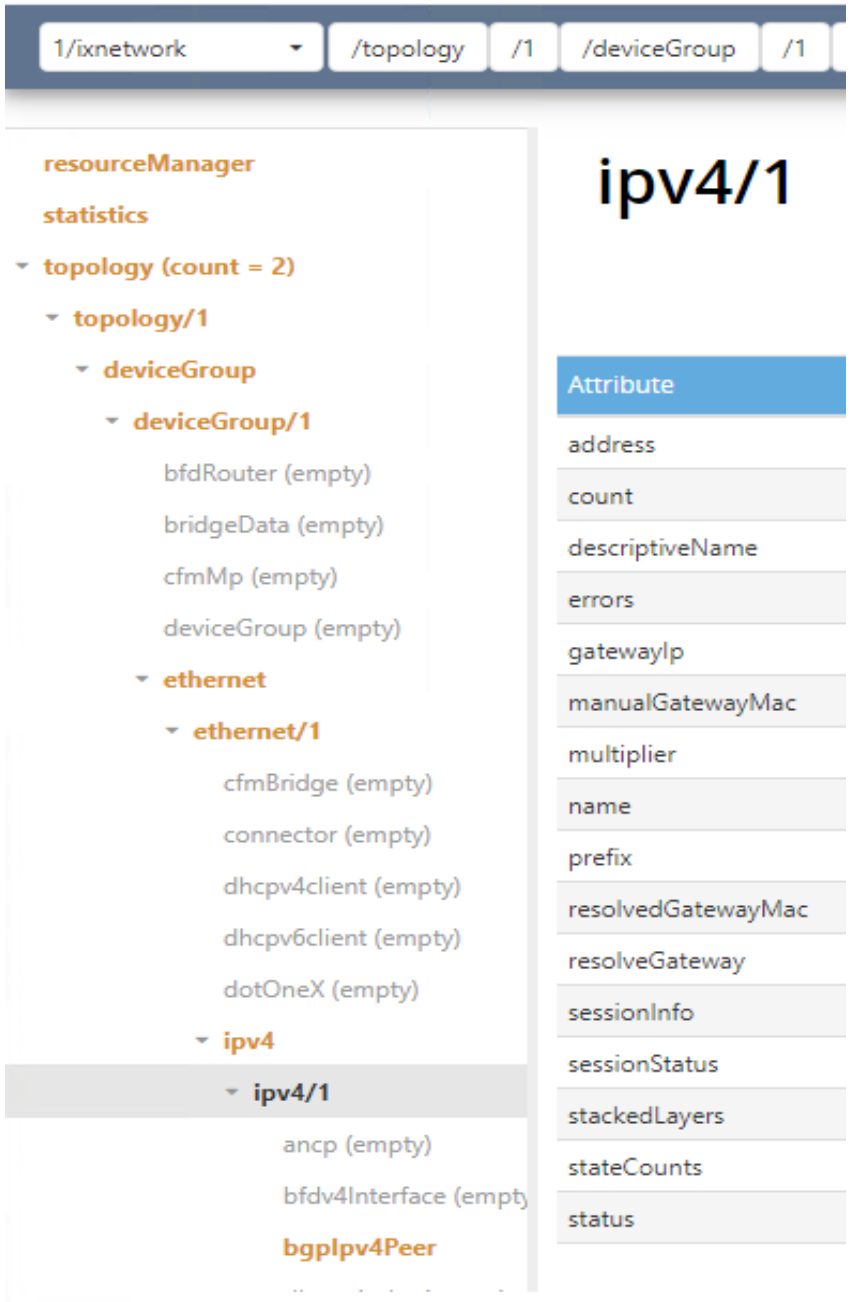
vPort1.Name = 'Topo1'

state = vPort1.State



Create an IPv4 Topology

```
10  ixNetwork.info('Creating Topology Group 1')
11  topology1 = ixNetwork.Topology.add(Name='Topo1', Ports=vPort1)
12
13  ixNetwork.info('Creating Device Group 1')
14  deviceGroup1 = topology1.DeviceGroup.add(Name='DG1', Multiplier='1')
15
16  ixNetwork.info('Creating Ethernet stack 1')
17  ethernet1 = deviceGroup1.Ethernet.add(Name='Eth1')
18
19  ixNetwork.info('Configuring IPv4')
20  ipv4 = ethernet1.Ipv4.add(Name='Ipv4')
21  ipv4.Address.Increment(start_value='10.1.1.1', step_value='0.0.0.1')
22
```



How NGPF Multi-Values Are Configured

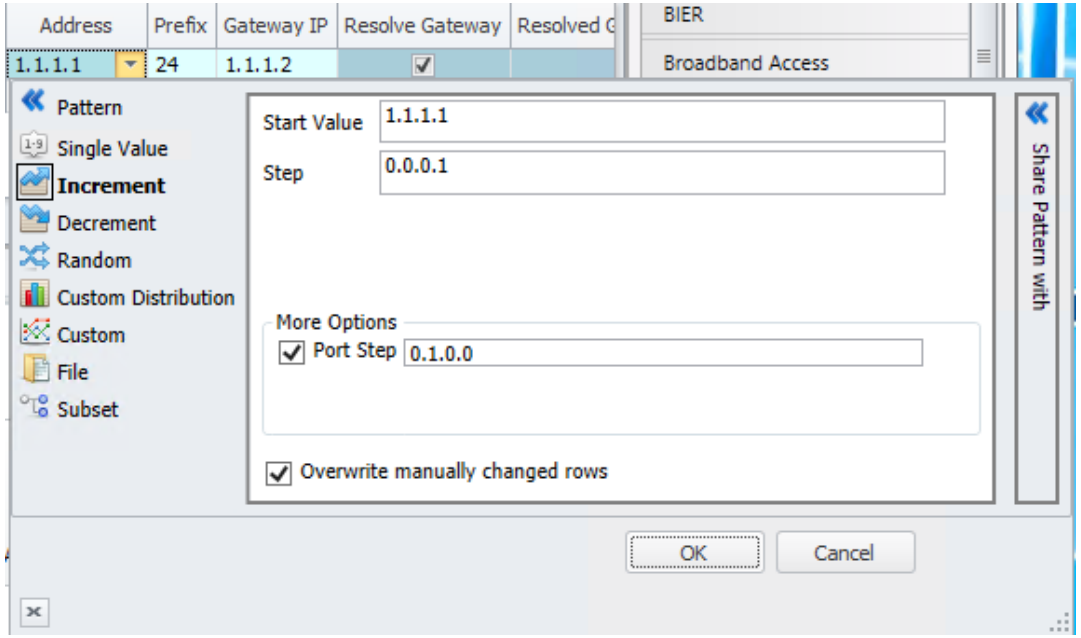
```
ipv4 = ethernet1.Ipv4.add(Name='Ipv4-2')

ipv4.Address.Increment(start_value='1.1.1.1',
                       step_value='0.0.0.1')
```

Multivalue choices:

- single, increment, decrement, custom, random
- More samples in RESTPy user guide

Multi-values in the GUI:



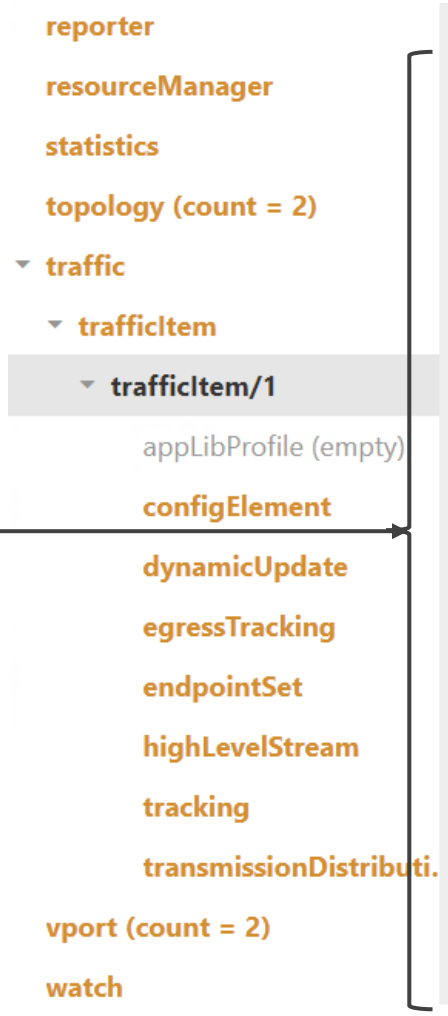
Creating Traffic Item

trafficItem = ixNetwork.Traffic.TrafficItem.**add**(Name='BGP Traffic', BiDirectional=False, TrafficType='ipv4')

print(trafficItem)

TrafficItem[0]: /api/v1/sessions/1/ixnetwork/traffic/trafficItem/1

AllowSelfDestined: False
BiDirectional: False
Enabled: True
Name: BGP Traffic
RouteMesh: oneToOne
SrcDestMesh: oneToOne
State: unapplied
Suspend: False
TrafficItemType: l2L3
TrafficType: ipv4



Attribute	Value
allowSelfDestined	false
biDirectional	false
egressEnabled	false
enabled	true
enableDynamicMplsLabelValues	false
errors	
flowGroupCount	1
frerDuplicateElimination	false
hasOpenFlow	false
hostsPerNetwork	1
interAsBgpPreference	one
interAsLdpPreference	two
maxNumberOfVpnLabelStack	2
mergeDestinations	true
multicastForwardingMode	replication

Modifying the Traffic Item

Traffic Items

#	Traffic Item	State	Enabled	Src/Dst Mesh	Route Mesh	Bi-Directional	Endpoint Set	Sources	Destinations
1	BGP Traffic	UNAPPLIED	☑	One - One	One - One	☐	EndpointSet-1	Topo1	Topo2

Modify traffic item properties

trafficItem.BiDirectional = True

trafficItem.Enabled = False

trafficItem.Name = 'BGP-Topo1'

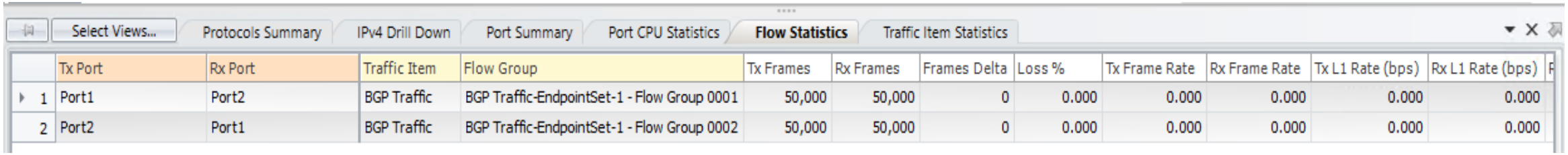
Modify multiple properties by using *update():*

configElement = trafficItem.ConfigElement.find()[0]

configElement.FrameRate.**update**(Type='percentLineRate', Rate=50)

configElement.TransmissionControl.**update**(Type='fixedFrameCount', FrameCount=10000)

Getting Stats



Select Views... | Protocols Summary | IPv4 Drill Down | Port Summary | Port CPU Statistics | **Flow Statistics** | Traffic Item Statistics

	Tx Port	Rx Port	Traffic Item	Flow Group	Tx Frames	Rx Frames	Frames Delta	Loss %	Tx Frame Rate	Rx Frame Rate	Tx L1 Rate (bps)	Rx L1 Rate (bps)
1	Port1	Port2	BGP Traffic	BGP Traffic-EndpointSet-1 - Flow Group 0001	50,000	50,000	0	0.000	0.000	0.000	0.000	0.000
2	Port2	Port1	BGP Traffic	BGP Traffic-EndpointSet-1 - Flow Group 0002	50,000	50,000	0	0.000	0.000	0.000	0.000	0.000

flowStatistics = session.**StatViewAssistant**('Flow Statistics')

print(flowStatistics)

STATS: **Row:0** View:Flow Statistics

Tx Port: Port1

Rx Port: Port2

Traffic Item: BGP Traffic

Flow Group: BGP Traffic-EndpointSet-1 - Flow Group 0001

Tx Frames: 50000

Rx Frames: 50000

Frames Delta: 0

Loss %: 0.000

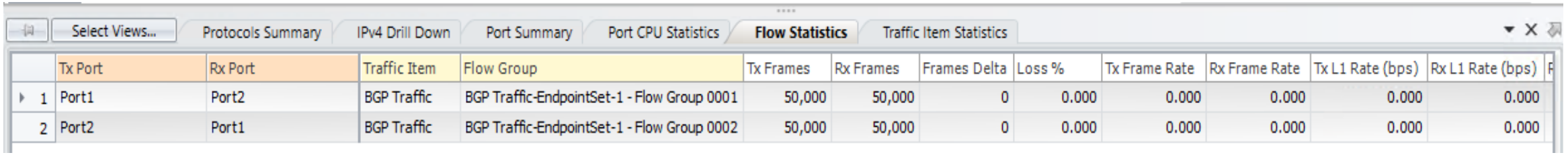
How to get stat values

flowStatistics.Rows[0]['Tx Frames']

flowStatistics.Rows[0]['Rx Frames']

Getting Stats

FILTER STATS WITH ADDROWFILTER()



	Tx Port	Rx Port	Traffic Item	Flow Group	Tx Frames	Rx Frames	Frames Delta	Loss %	Tx Frame Rate	Rx Frame Rate	Tx L1 Rate (bps)	Rx L1 Rate (bps)
1	Port1	Port2	BGP Traffic	BGP Traffic-EndpointSet-1 - Flow Group 0001	50,000	50,000	0	0.000	0.000	0.000	0.000	0.000
2	Port2	Port1	BGP Traffic	BGP Traffic-EndpointSet-1 - Flow Group 0002	50,000	50,000	0	0.000	0.000	0.000	0.000	0.000

flowStatistics = session.**StatViewAssistant**('Flow Statistics')

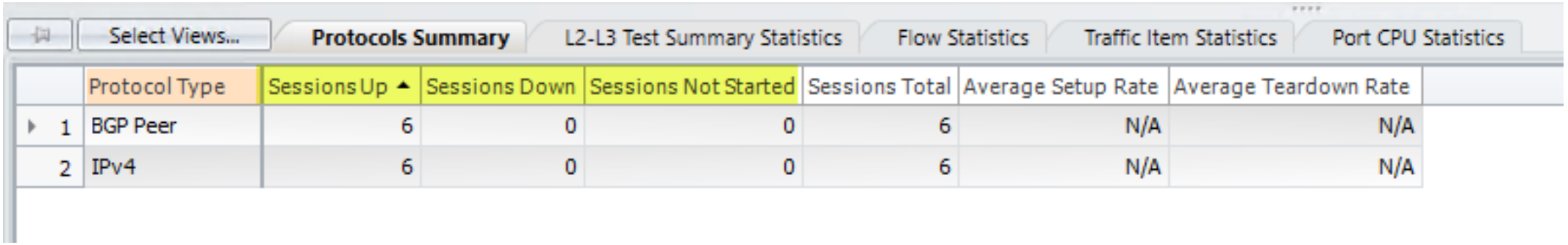
flowStatistics.**AddRowFilter**('Rx Port', flowStatistics.REGEX, '^Port2$'))

Use LESS_THAN, GREATER_THAN, EQUAL, LESS_THAN_OR_EQUAL, GREATER_THAN_OR_EQUAL

flowStatistics.**AddRowFilter**('Loss %', flowStatistics.GREATER_THAN, "5")

Getting Stats

VERIFYING STATS WITH CHECKCONDITION()



	Protocol Type	Sessions Up	Sessions Down	Sessions Not Started	Sessions Total	Average Setup Rate	Average Teardown Rate
1	BGP Peer	6	0	0	6	N/A	N/A
2	IPv4	6	0	0	6	N/A	N/A

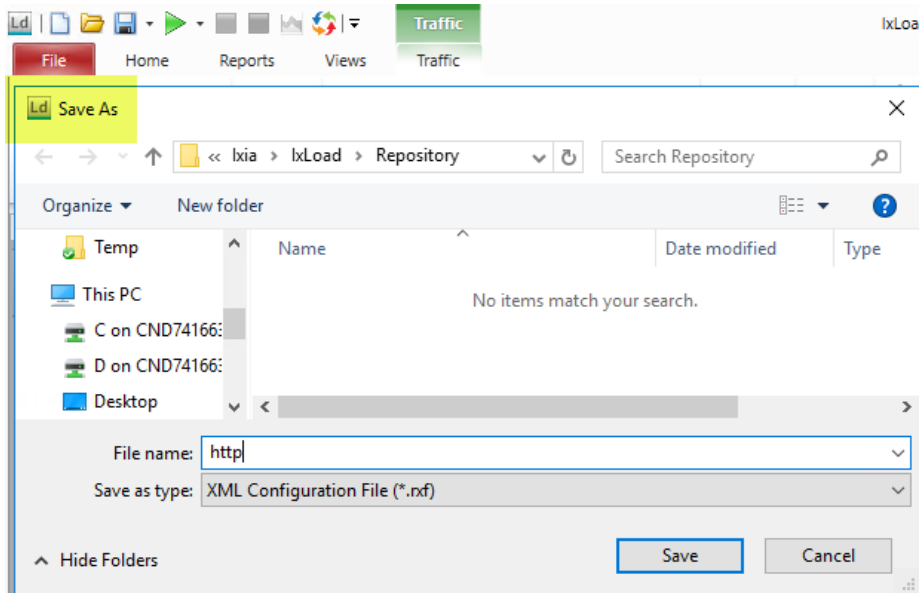
```
protocolSummary = session.StatViewAssistant('Protocols Summary')
protocolSummary.CheckCondition('Sessions Not Started', protocolSummary.EQUAL, 0)
protocolSummary.CheckCondition('Sessions Down', protocolSummary.EQUAL, 0)
ixNetwork.info(protocolSummary)
```

Fastest Way to Automate

LOAD A SAVED CONFIG FILE

- Create your configuration on the GUI
- Make sure the configuration is working
- Release all ports and delete the chassis/ports. Leaving just the configuration.
- Save the configuration as an .ixncfg file.
- Use the sample script from github called **loadConfigFile.py** to:

  Set the chassis IP and portList to use.
  Load the saved config file
  Assign ports
  Start all protocols
  Verify all protocols
  Start traffic
  Get stats



Note: The saved config version is compatible on future version releases, but it is not compatible in prior released versions. For example. A 9.0 saved config file will not work in IxNetwork 8.50.

Use Find() to Query for Objects

Traffic Items

#	Traffic Item	State	Enabled	Src/Dst Mesh	Route Mesh	Bi-Directional	Endpoint Set	Sources	Destinations
1	BGP Traffic	UNAPPLIED	✓	One - One	One - One		EndpointSet-1	Topo1	Topo2
2	BGP Traffic 2	UNAPPLIED	✓	One - One	One - One		EndpointSet-1	Topo1	Topo2
3	BGP Traffic 3	UNAPPLIED	✓	One - One	One - One		EndpointSet-1	Topo1	Topo2
4	OSPF	UNAPPLIED	✓	One - One	One - One		EndpointSet-1	Topo1	Topo2
5	ISIS	UNAPPLIED	✓	One - One	One - One		</> EndpointSet-1	Topo1	Topo2

Use regular expression with find().

```
trafficItem = ixNetwork.Traffic.TrafficItem.find(Name='BGP Traffic')
trafficItem = ixNetwork.Traffic.TrafficItem.find(Name='^BGP Traffic$')
trafficItem = ixNetwork.Traffic.TrafficItem.find(Name=('BGP Traffic 2|OSPF'))
```

```
trafficItem = ixNetwork.Traffic.TrafficItem.find()[0]   # Get the first object in the list
trafficItem = ixNetwork.Traffic.TrafficItem.find()      # Get the last object in the list
```

https://github.com/OpenIxia/IxNetwork/tree/master/RestPy/SampleScripts

Branch: master ▾ **IxNetwork** / RestPy / **SampleScripts** /

Create new file | Upload files | Find file | History



hubertgee Removed and replaced saved config file that has chassis/ports saved — Latest commit 2697f84 15 days ago

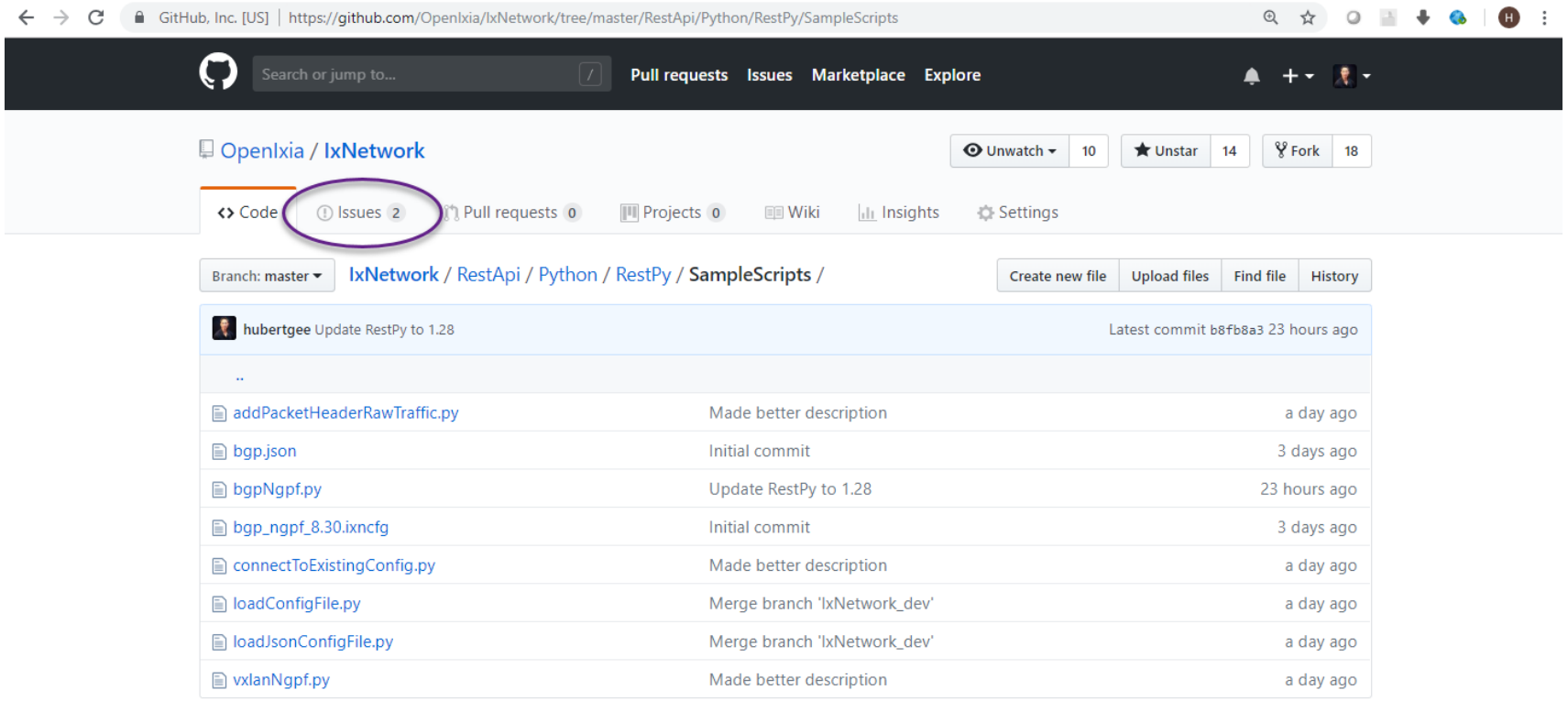
..		
QuickTestNgpf_vm8.20.ixncfg	Initial commit for loadQuickTest.py	14 months ago
bgp.json	Moved RestPy	2 years ago
bgpNgpf.py	New saved config file with no chassis/ports saved	15 days ago
bgpNgpf.robot	Fixed typo	12 months ago
bgp_ngpf_8.50.json	Config license after loading the config file	16 months ago
bgp_ngpf_9.00.ixncfg	New saved config file with no chassis/ports saved	15 days ago
connectToExistingConfig.py	Fixed type on UserName parameter	2 months ago
createTrafficItemAddPacketHeader.py	Fixed type on UserName parameter	2 months ago
egressTracking.py	Fixed type on UserName parameter	2 months ago
evpnNgpf.py	Fixed type on UserName parameter	2 months ago
igmpHostAndQuerier.py	Initial sample script: igmp host and querier	8 months ago
loadConfigFile.py	Fixed type on UserName parameter	2 months ago
loadConfigFile.robot	New saved config file with no chassis/ports saved	15 days ago
loadJsonConfigFile.py	Fixed type on UserName parameter	2 months ago
loadQuickTest.py	Fixed type on UserName parameter	2 months ago

Submitting Questions and Issues Regarding RESTPy

To ask RESTPy questions, enhancements and bugs, use our Github/OpenIxia/IxNetwork project site.

In the IxNetwork repository, go inside “issues”:

- https://github.com/OpenIxia/IxNetwork



RESTPy Resources

Requirement

- Minimum of IxNetwork 8.52
- Python 2.7 and 3+
- Python requests module

Support

- IxNetwork NGPF and Classic framework
- On Windows, Windows Connection Mgr and Linux

Installation

- pip install ixnetwork_restpy
- From Github: https://github.com/OpenIxia/ixnetwork_restpy

Sample scripts: https://github.com/OpenIxia/IxNetwork/tree/master/RestPy/SampleScripts

User guide: https://openixia.github.io/ixnetwork_restpy


KEYSIGHT
TECHNOLOGIES